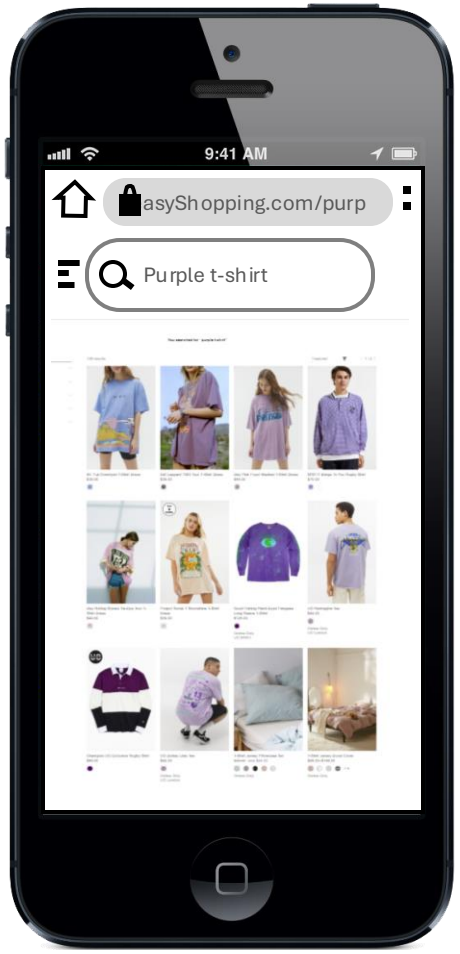


PROJECT 2
Labelling

SO: Sessió d'Orientació

Project 2 - Part 1

Artificial intelligence

2025-2026

Universitat Autònoma de Barcelona

1.Introducció

En aquesta pràctica, resoldrem un problema simple **d'etiquetatge d'imatges**.
Donat un conjunt d'imatges d'un catàleg de roba, desenvoluparem algorismes per aprendre com etiquetar imatges per tipus i color.
Per simplificar la pràctica, utilitzarem un conjunt petit d'etiquetes:

8 tipus de roba i 11 colors bàsics.

El sistema s'executarà en una imatge donada i retornarà una etiqueta per a un tipus de roba i una o més etiquetes per a colors. Implementareu els següents algorismes vistos a classe de teoria:

1. K-NN (k−nn): **Mètode de classificació supervisada per classificar els tipus de roba.**
2. K-means (k-means): **Mètode de classificació no supervisada per trobar colors predominants.**

En aquesta **2a part del Projecte,** ens centrarem en: **l'algorisme K-NN**

2. Fitxers requerits

Hem de tenir descarregades les següents carpetes:

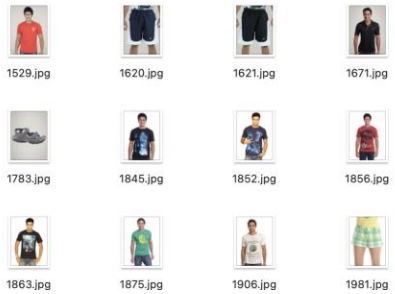
1. **Images:** Carpeta que conté el set d'imatges que utilitzarem.

   A. gt.json: Fitxer que conté informació sobre la classe d'imatges a entrenar (Train).

   B. Train: Carpeta amb el set d'imatges que utilitzarem com a set d'entrenament. Sobre elles **(train set)** trobarem informació sobre quina classe pertany en el fitxer **gt.json.**

   C. Test: Carpeta amb el set d'imatges que volem etiquetar i de les quals no tenim informació, aquest serà el nostre set d'experimentació **(test set).**

Classes de Train data estan etiquetades per tipus I colors



Conjunt d'imatges de les carpetes Train/Test folders

2. Fitxers requerits

Per dur a terme la pràctica, hauràs de descarregar les següents carpetes:

2. 📁 Test: Carpeta que conté el conjunt de fitxers necessaris per poder realitzar les proves sol·licitades en els scripts de prova (no els has d'utilitzar en els teus scripts, les funcions de prova els carreguen automàticament en el setUp).

test_cases_knn.pkl

3. utils.py: Conté un seguit de **functions** necessaries per convertir les imatges a color en altres espais, bàsicament les convertim a escala de grisos (grey-level) rgb2gray() i aconseguim els 11 colors bàsics get_color_prob().

4. utils_data.py: Conté un seguit de **functions** necessaries per obrir i tractar les imatges. Com per exemple llegir les imatges i etiquetes amb read_dataset, .

4. KNN.py: Fitxer a on programareu les funcions necessaries per implementar K-NN per l'etiquetatge automàtic de forma.

5. TestCases_knn.py: Arxiu amb el qual podreu comprovar si les funcions que programeu en el fitxer KNN.py donen el resultat esperat.

3. Preliminars

En aquesta pràctica treballarem amb **matrius** per a imatges i també amb **centròids** que utilitzen algorismes iteratius. Per implementar aquests algorismes de manera eficient, et recomanem que tinguis un bon domini de la **llibreria Numpy** per simplificar les teves funcions.

Els següents enllaços contenen alguns exercicis bàsics de Numpy que et poden ajudar a comprendre com funciona aquesta llibreria:

- NumPy Bàsic (41 exercicis amb solució): https://www.w3resource.com/python-exercises/numpy/basic/index.php

- NumPy arrays (192 exercicis amb solució): https://www.w3resource.com/pythonexercises/numpy/index-array.php

- NumPy Sorting and Searching (8 exercicis amb solució): https://www.w3resource.com/pythonexercises/numpy/python-numpy-sorting-and-searching.php

- Altres exercicis: https://www.w3resource.com/python-exercises/numpy/index.php

3. Preliminars

Al final dels tutorials, hauries de ser capaç de realitzar les següents operacions amb matrius numpy:

- Redimensionar una matriu
- Calcular la mitjana dels valors d'un array
- Realitzar operacions entre matrius i vectors (per exemple, restar un vector a cada fila d'una matriu)
- Realitzar les dues operacions anteriors només amb certes files d'una matriu.

```
##################################################
# Code determining the pixels that belong to each
# of the six clusters for 1000 80x60 images.
##################################################

import time
import random import numpy as np K=6
N = 80*60 Iteracions = 1000

# Version with python lists.
###########################
list = random.choices(range(K), k=N)
t0 = time.time()
grups = {}
for iteration in range(Iterations):
    grups = {}
    for k in range(K):
        grups[k] = [True if elem == k else False for elem
in list]
t1 = time.time()-t0
# Version with numpy matrix
###########################
vector_np = np.random.randint(K, N)
t0 = time.time()
for iteration in range(Iterations):
    grups = {}
    for k in range(K):
        grups[k] = vector_np==k
t2 = time.time()-t0

print(f'time to obtain the {K} groups for {N} pixels,
{Iterations} times.')
print(f'        WITHOUT numpy:\t{t1} seconds')
print(f'        WITH numpy:\t\t{t2} seconds')
print(f'WITH is {t1/t2} times faster that WITHOUT numpy')
```

The execution gives:

```
        WITHOUT numpy: 4.341734886169434 seconds
        WITH numpy: 0.006999969482421875 seconds
WITH is 620.2505449591281 times faster than WITHOUT
numpy.
```

3. Preliminars

Abans de començar a programar, és molt recomanable entendre la classe amb la qual anem a treballar:

KNN

KNN És una classe dissenyada per carregar totes les dades, transformar-les i executar l'algorisme KNN

Classe KNN:

Atributs

```
self.labels = np.array(labels)
self.train_data = np.random.randint(8, size=[10, 4800])
self.neighbors = np.random.randint(k, size=[test_data.shape[0], k])
```

3. Preliminars

Abans de començar a programar, és molt recomanable entendre la classe amb la qual anem a treballar:

KNN

KNN És una classe dissenyada per carregar totes les dades, transformar-les i executar l'algorisme KNN

Classe KNN:

Funcions:

```
def _init_train(self, train_data):    inicialitzar dades d'entrenament
def get_k_neighbours(self, test_data, k):  càlcul dels veins més propers
def get_class(self): assignar la classe segons la majoria de vots
def predict(self, test_data, k): prediu la classe al qual cada imatge pertany
```

4. Què és el que hem de programar?

Has de codificar l'algorisme de classificació K-NN per poder etiquetar la forma de cada imatge.

Haureu de realitzar quatre tasques:

4.1. Funció per **inicialitzar K-NN: _init_train**

4.2. Funció que troba els k veïns més propers: **get_k_neighbours**

4.3. Funció que selecciona l'etiqueta que més apareix als veïns: **get_class**

4.4. Funció que assigna l'etiqueta de classe a les dades de test: **predict**

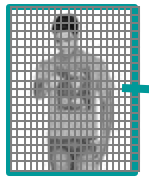
4.1. Funció per **inicialitzar K-NN: _init_train**

Sessió de Teoria

Definició d'espai de característiques

1) Eliminarem el COLOR, ja que no el necessitem per representar la forma

2) Agafarem directament els píxels de la imatges com la característica de cada posició de la imatge.



(1,0,0.5, ... ,1) -> Vector de 1x4800 on 4800=80x60 píxels

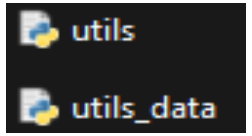
Sessió pràctica

`_init_train`: Funció que s'assegura que la variable `train_data` de la classe `KNN`, la qual conté el nostre conjunt d'entrenament, tingui el format `float`, i tot seguit n'extreu les característiques

`train_data`: dimensions de N x 4800, on

- `N`: nombre d'imatges de `train_data`
- `4800`: nombre de píxels d'una imatge

Recordeu!

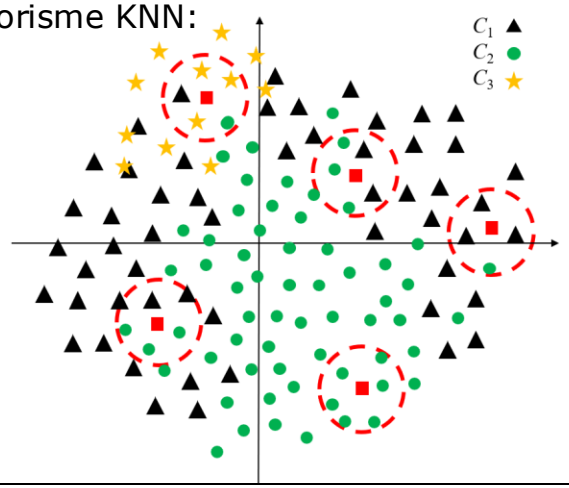


`rgb2gray()`

4.2. Funció que troba els k veins més propers: **get_k_neighbours**

Sessió de Teoria

L'algorisme KNN:




```
Funció Classificar(y⃗)
1.    Per (x⃗^j ∈ X) fer L=inserir([d(y⃗,x⃗^j),C_i],L)Fper
2.    veïns=Primers_k(ordenar_d(L))
3.    Si(comptar(veïns,C_1)>comptar(veïns,C_2))
      llavors
4.     y⃗ ∈ C_1
5.    Sinó y⃗ ∈ C_2
6.    Fsi
FFunció
```

Sessió pràctica

`get_k_neighbours:` Funció que pren com a entrada el conjunt de test que volem etiquetar (`test_data`) i fa el següent:

1. Canvia les dimensions de les imatges de la mateixa manera que ho hem fet amb el conjunt d'entrenament. (`test_data`)
2. Calcula la distància entre les mostres del `test_data` i les del `train_data`.
3. Guarda a la variable de classe `self.neighbors` les K etiquetes de les imatges més pròximes per a cada mostra del test.

Important: El càlcul de distàncies és molt costós si es fa en un bucle.
Es recomana usar la funció `cdist`, de la llibreria `scipy.spatial.distance`,

4.2. Funció que troba els k veins més propers: **get_k_neighbours**

Sessió de Teoria

Exemple:

$\vec{y}$ =[1,1,1,0.2,0.5,0,0,1,… ,,1]

$$\begin{pmatrix} x_1^1 & \dots & x_{4800}^1 C_1 \\ \vdots & \ddots & \vdots \quad \vdots \\ x_1^{n_1} & \dots & x_{4800}^{n_1} C_1 \\ x_1^1 & \dots & x_{4800}^1 C_2 \\ \vdots & \ddots & \vdots \quad \vdots \\ x_1^{n_2} & \dots & x_{4800}^{n_2} C_2 \\ & \vdots \quad \vdots & \vdots \\ x_1^1 & \dots & x_{4800}^1 C_k \\ \vdots & \ddots & \vdots \quad \vdots \\ x_1^{n_k} & \dots & x_{4800}^{n_k} C_k \end{pmatrix}$$

d=23

[1,1,1,0.1,0.4,0.2,1,1,… ,1]

d=59

[0,0,0,0.7,0.5,1,0,1,… ,1]

d=103

[0,0,0,0.2,1,1,0,0.4,… ,0]

Funció Classificar($\vec{y}$)
1. **Per** ($\vec{x}^j \in \mathbf{X}$) fer L=inserir($\lfloor d(\vec{y},\vec{x}^j), C_j \rfloor$,L)**Fper**
2. veïns=Primers_k(ordenar_d(L))
3. **Si**(comptar(veïns,C_1)>comptar(veïns,C_2)) **llavors**
4. $\vec{y} \in C_1$
5. **Sinó** $\vec{y} \in C_2$
6. **Fsi**

FFunció

Sessió pràctica

Més detalls:

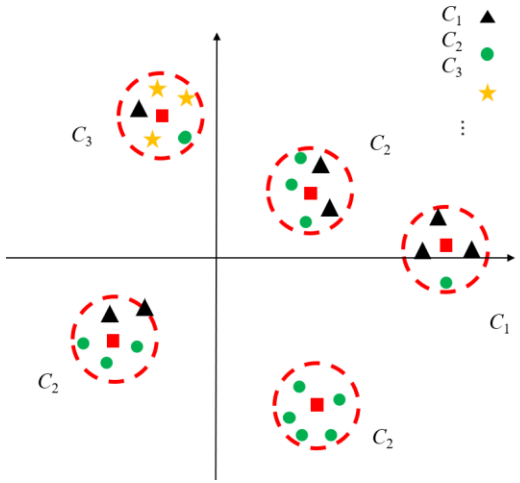
get_k_neighbours: Funció que pren com a entrada el conjunt de test que volem etiquetar (test_data) i fa el següent:

- **Input:** test_data i k
- **Retorna:** self.neighbors

```
def get_k_neighbours(self,test_data,k)
: param test_data: array that has to be
          shaped to a NxD matrix
          (N points in a D dimensional
space)
: param k: number of neighbors to look at
: return: the matrix self.neighbors of NxK
          where
          ij-th entry is the j-th nearest
          train point to the i-th test
point
```

4.3. Funció que selecciona l'etiqueta que més apareix als veins: **get_class**

Sessió de Teoria




```
Funció Classificar(y⃗)
1.    Per (x⃗^j ∈ X) fer L=inserir([d(y⃗,x⃗^j),C_j],L)Fper
2     veïns=Primers_k(ordenar_d(L))
3.    Si(comptar(veïns,C_1)>comptar(veïns,C_2))
      llavors
4.     y⃗ ∈ C_1
5.    Sinó y⃗ ∈ C_2
6.    Fsi
FFunció
```

Sessió pràctica

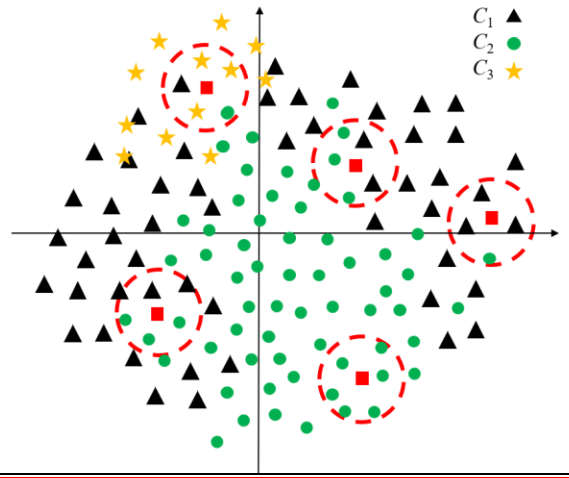
`get_class`: Funció que comprova quina és l'etiqueta que més vegades ha aparegut a la variable de classe neighbors per a cada imatge del conjunt de test (`test_data`),

Retorna un array amb aquesta classe per a cada imatge. Aquest array tindrà tants elements com punts s'hauran entrat a la funció `predict`.

- **Input:** -
- **Retorna:** array class_labels

4.4 Funció que assigna l'etiquesta de classe a les dades de test: **predict**

Sessió de Teoria




```
Funció Classificar(y⃗)
1.    Per (x⃗^j ∈ X) fer L=inserir(|d(y⃗,x⃗^j),C_j|,L)Fper
2.    veïns=Primers_k(ordenar_d(L))
3.    Si(comptar(veïns,C_1)>comptar(veïns,C_2))
      llavors
4.     y⃗ ∈ C_1
5.    Sinó y⃗ ∈ C_2
6.    Fsi
FFunció
```

Sessió pràctica

`predict`: Funció que pren com a entrada el conjunt de test que volem etiquetar (`test_data`) i el nombre de veïns que volem tenir en compte (`k`), busca els seus veïns utilitzant

- **Input:** test_data, k
- **Retorna:** array class_labels

Aquesta funció bàsicament crida a les dues anteriors

`get_k_neighbours`

`get_class`

5. Avaluació per qüestionaris

Per a l'avaluació d'aquesta part de la pràctica haureu de contestar un seguit de qüestionaris del Campus Virtual.

- **Proj2.1-Q1**: qüestionari corresponent a la **SO** amb preguntes tipus test i avaluació de les funcions *_init_train, get_k_neughbours, get_class i predict.*
- **Proj2.1-Q2**: qüestionari corresponent a la **SS** amb preguntes tipus test i avaluació on haureu d'executar l'algoritme KNN amb imatges externes.
- **Proj2-Par1**: des de Divendres, 20 de Març 17:00h fins Diumenge, 22 de Març 23:55h. Avaluació de totes les funcions anteriors.

ATENCIÓ! és important que tingueu en compte els següents punts:

1. La **correcció** del codi es fa de manera **automàtica**, per tant, assegureu-vos de penjar els mètodes amb la nomenclatura i format correctes. Si no ho poseu bé la nota de l'exercici serà un 0.

2. El codi està sotmès a **detecció automàtica** de plagis durant la correcció.

3. Per evitar que el codi entri en bucles infinits hi ha un **límit de temps** per a cada exercici, per tant si les vostres funcions triguen massa les considerarà incorrectes.

Recordeu el que es diu a la **guia docent** sobre **copiar o deixar copiar ....**

Sense perjudici d'altres mesures disciplinàries que s'estimin oportunes, i d'acord amb la normativa acadèmica vigent, les **irregularitats comeses per l'alumnat** que puguin conduir a una variació de la qualificació es qualificaran amb un zero (0). Les activitats d'avaluació qualificades d'aquesta forma i per aquest procediment no seran recuperables. Si és necessari superar qualsevol d'aquestes activitats d'avaluació per aprovar l'assignatura, aquesta assignatura quedarà suspesa directament, sense oportunitat de recuperar-la en el mateix curs. Aquestes irregularitats inclouen, entre d'altres:

- còpia total o parcial d'una pràctica, informe, o qualsevol altra activitat d'avaluació;
- deixar copiar;
- ús no autoritzat i/o no referenciat de la IA (p. ex,Copilot, ChatGPToequivalents) per a resoldre exercicis, pràctiques i/o qualsevol altra activitat avaluable;
- presentar un treball de grup no fet íntegrament pels membres del grup;
- presentar com a propis materials elaborats per un tercer, encara que siguin traduccions o adaptacions, i en general treballs amb elements no originals i exclusius de l'alumnat.
- tenir dispositius de comunicació (com telèfons mòbils, smart watches, etc.) accessibles durant les proves d'avaluació teòric-pràctiques individuals (exàmens).

En resum: **copiar, deixar copiar o plagiar** en qualsevol de les activitats d'avaluació equival a un SUSPENS amb **nota inferior o igual a 3,0.**